

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЁТ
к лабораторной работе №5
на тему
«УПРАВЛЕНИЕ ПРОЦЕССАМИ И ВЗАИМОДЕЙСТВИЕ ПРОЦЕССОВ»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Постановка задачи	3
2	Выполнение работы	4
2.1	Архитектура программы	4
2.2	Реализация вычисления сигнатуры	5
2.3	Сборка, запуск и демонстрация работы	5
	Вывод	6
	Список использованных источников	7
	Приложение А (обязательное) Листинг программного кода	8

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является изучение механизмов управления процессами и взаимодействия процессов в операционной системе *Linux*.

В рамках варианта №5 необходимо разработать программу параллельного анализатора файловой системы, которая:

- выполняет рекурсивный обход заданной директории;
- для каждого найденного регулярного файла вычисляет сигнатуру (результат операции *XOR* всех байтов файла);
- использует многопроцессорную архитектуру для ускорения обработки;
- ограничивает количество одновременно работающих процессов-агентов;
- агрегирует результаты работы агентов в отдельном процессе-коллекторе через программный канал (*pipe*).

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Архитектура программы

Программа реализована на языке *C* с использованием стандартов *POSIX*. Архитектура системы состоит из трех основных типов компонентов, взаимодействующих через системные вызовы управления процессами и межпроцессного взаимодействия [1].

2.1.1 Главный процесс

Координирует запуск системы. Он выполняет следующие действия:

- обрабатывает аргументы командной строки (путь к директории и максимальное количество агентов);
- создает программный канал (*pipe*) для передачи данных от агентов к коллектору [1];
- порождает процесс-коллектор через *fork* [2];
- запускает управляющий модуль (*Manager*) в контексте основного процесса;
- ожидает завершения всех процессов перед выходом [3].

2.1.2 Управляющий модуль

Реализует логику обхода файловой системы и управления пулом процессов.

- Рекурсивно обходит дерево директорий, используя функции *opendir* и *readdir* [1].
- Для каждой директории порождает новый процесс-агент.
- Контролирует количество активных дочерних процессов с помощью системного вызова *waitpid* с флагом *WNOHANG* или блокирующего ожидания, если достигнут лимит *max_agents* [3].
- Обеспечивает параллельную обработку различных ветвей файловой системы.

2.1.3 Процесс-агент

Выполняет непосредственную обработку файлов в назначенной ему директории.

- Перебирает регулярные файлы в директории.
- Для каждого файла вычисляет *XOR*-сигнатуру, вызывая модуль *signature*.
- Формирует строку результата, содержащую путь к файлу, значение сигнатуры, временную метку последнего изменения и свой *PID*.
- Передает сформированную строку в коллектор через дескриптор записи программного канала [4].
- Завершает работу после обработки всех файлов в своей директории.

2.1.4 Процесс-коллектор

Отвечает за сбор и вывод результатов.

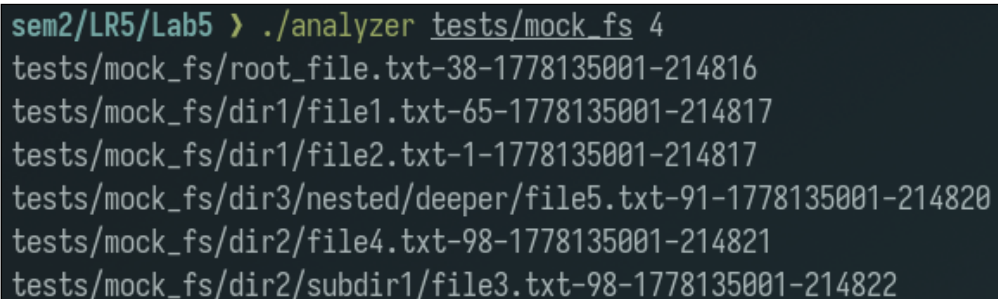
- Читает данные из программного канала, пока он не будет закрыт всеми пишущими процессами.
- Выводит полученную информацию в стандартный поток вывода (*stdout*).
- Использование отдельного процесса для вывода позволяет избежать перемешивания строк от разных агентов (атомарность записи в *pipe* в пределах *PIPE_BUF*) и предотвращает блокировки пишущих процессов при переполнении буфера канала.

2.2 Реализация вычисления сигнатуры

Модуль *signature* реализует функцию `calculate_xor_signature`, которая открывает файл в бинарном режиме и последовательно применяет операцию исключающего ИЛИ ко всем байтам содержимого. Для оптимизации чтения используется буфер размером 4096 байт.

2.3 Сборка, запуск и демонстрация работы

Проект собирается с помощью утилиты `make`. Исполняемый файл `analyzer` принимает два аргумента:



```
sem2/LR5/Lab5 > ./analyzer tests/mock_fs 4
tests/mock_fs/root_file.txt-38-1778135001-214816
tests/mock_fs/dir1/file1.txt-65-1778135001-214817
tests/mock_fs/dir1/file2.txt-1-1778135001-214817
tests/mock_fs/dir3/nested/deeper/file5.txt-91-1778135001-214820
tests/mock_fs/dir2/file4.txt-98-1778135001-214821
tests/mock_fs/dir2/subdir1/file3.txt-98-1778135001-214822
```

Рисунок 2.1 – Результат выполнения программы параллельного анализатора файловой системы

На рисунке 2.1 приведена демонстрация работы программы при анализе тестовой директории с ограничением в 4 параллельных агента.

ВЫВОД

В ходе выполнения лабораторной работы были изучены механизмы создания процессов с помощью *fork*, синхронизации процессов через семейство функций *wait* и организации межпроцессного взаимодействия с использованием неименованных каналов (*pipe*) [1]. Была реализована эффективная многопроцессорная архитектура, позволяющая параллельно обрабатывать данные файловой системы с контролем нагрузки на систему. Закреплены навыки работы с низкоуровневыми *POSIX API* и обеспечения безопасности при работе с разделяемыми ресурсами (дескрипторами файлов).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Linux manual pages: fork(2), pipe(2), waitpid(2), opendir(3) [Электронный ресурс]. – Режим доступа: <https://man7.org/linux/man-pages/>. – Дата доступа: 12.04.2026.

[2] Anatomy of Linux process management [Электронный ресурс]. – Режим доступа: <https://developer.ibm.com/tutorials/l-linux-process-management/>. – Дата доступа: 12.04.2026.

[3] Exit status of a child process in Linux [Электронный ресурс]. – Режим доступа: <https://www.geeksforgeeks.org/linux-unix/exit-status-child-process-linux/>. – Дата доступа: 12.04.2026.

[4] Beej's Guide to Interprocess Communication: Pipes [Электронный ресурс]. – Режим доступа: <https://beej.us/guide/bgipc/html/split/pipes.html>. – Дата доступа: 12.04.2026.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Код файла main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include "manager.h"
#include "collector.h"

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <directory> [max_agents]\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    const char *directory = argv[1];
    int max_agents = 4;

    if (argc >= 3) {
        max_agents = atoi(argv[2]);
        if (max_agents <= 0) {
            fprintf(stderr, "Invalid max_agents value. Must be > 0.\n");
            exit(EXIT_FAILURE);
        }
    }

    int pipefd[2];
    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    pid_t collector_pid = fork();
    if (collector_pid == -1) {
        perror("fork collector");
        exit(EXIT_FAILURE);
    }

    if (collector_pid == 0) {
        run_collector(pipefd[0], pipefd[1]);
        exit(EXIT_SUCCESS);
    } else {
        close(pipefd[0]);

        run_manager(directory, max_agents, pipefd[1]);

        close(pipefd[1]);

        if (waitpid(collector_pid, NULL, 0) == -1) {
            perror("waitpid");
        }
    }
}
```



```

        exit(EXIT_FAILURE);
    }
}

return EXIT_SUCCESS;
}

```

Листинг А.2 – Код файла manager.c

```

#include "manager.h"

#include "agent.h"

#include <dirent.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

static void wait_for_one_child(int* active_agents) {
    int status = 0;
    pid_t pid = -1;

    do {
        pid = waitpid(-1, &status, 0);
    } while (pid == -1 && errno == EINTR);

    if (pid > 0) {
        (*active_agents)--;
        return;
    }

    if (pid == -1 && errno == ECHILD) {
        *active_agents = 0;
    }
}

static void traverse_dir(const char* dirpath, int max_agents, int
write_fd, int* active_agents) {
    while (*active_agents >= max_agents) {
        wait_for_one_child(active_agents);
    }

    (*active_agents)++;
    pid_t pid = fork();
    if (pid == 0) {
        run_agent(dirpath, write_fd, -1);
        _exit(EXIT_FAILURE);
    }

    if (pid < 0) {
        (*active_agents)--;
    }
}

```

```

        return;
    }

    DIR* dir = opendir(dirpath);
    if (dir == NULL) {
        return;
    }

    struct dirent* entry = NULL;
    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name,
"..") == 0) {
            continue;
        }

        size_t path_len = strlen(dirpath) + 1 + strlen(entry->d_name)
+ 1;
        char* full_path = malloc(path_len);
        if (full_path == NULL) {
            continue;
        }

        snprintf(full_path, path_len, "%s/%s", dirpath, entry->d_name);

        struct stat st;
        if (lstat(full_path, &st) == 0 && S_ISDIR(st.st_mode)) {
            traverse_dir(full_path, max_agents, write_fd, active_agents);
        }

        free(full_path);
    }

    closedir(dir);
}

void run_manager(const char* root_dir, int max_agents, int write_fd) {
    if (root_dir == NULL) {
        return;
    }

    if (max_agents < 1) {
        max_agents = 1;
    }

    int active_agents = 0;
    traverse_dir(root_dir, max_agents, write_fd, &active_agents);

    while (active_agents > 0) {
        wait_for_one_child(&active_agents);
    }
}

```

Листинг А.3 – Код файла agent.c

```
#include "agent.h"
```

```

#include "signature.h"

#include <dirent.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <unistd.h>

void run_agent(const char* dirpath, int write_fd, int read_fd_to_close) {
    if (read_fd_to_close >= 0) {
        close(read_fd_to_close);
    }

    DIR* dir = opendir(dirpath);
    if (dir == NULL) {
        close(write_fd);
        exit(EXIT_SUCCESS);
    }

    struct dirent* entry = NULL;
    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name,
"..") == 0) {
            continue;
        }

        size_t path_len = strlen(dirpath) + 1 + strlen(entry->d_name)
+ 1;
        char* full_path = malloc(path_len);
        if (full_path == NULL) {
            continue;
        }

        snprintf(full_path, path_len, "%s/%s", dirpath, entry->d_name);

        struct stat st;
        if (lstat(full_path, &st) == 0 && S_ISREG(st.st_mode)) {
            unsigned char signature = calculate_xor_signature(full_path);
            long timestamp = (long)st.st_mtime;
            pid_t agent_pid = getpid();

            dprintf(write_fd, "%s-%u-%ld-%d\n", full_path, signature,
timestamp, (int)agent_pid);
        }

        free(full_path);
    }

    closedir(dir);
    close(write_fd);
    exit(EXIT_SUCCESS);
}

```